

RFC-0003

Arquitetura Antifraude e Validação de Integridade

(Geolocalização + Biometria + Dispositivo + Identidade Segura)

Status: Aprovado e Atualizado (v1.1)

Autor: Equipe de Arquitetura

Data: 07/05/2026

Depende de: RFC-0001, RFC-0002, RFC-0005, Arquitetura de Identidade e Acesso Segura

1. Contexto

O sistema registra presença com base em:

- Geolocalização (Lat/Lng contra o Polígono da Unidade).
- Biometria facial e Prova de Vida (*Liveness*).
- Identidade Canônica Governamental (Validação de CPF).
- Nível de Confiança do Dispositivo.
- Contexto organizacional (Regras de Negócio).

Sem mecanismos antifraude severos, o sistema de controle de presença se torna vulnerável a:

- GPS spoofing (Falsificação de coordenadas).
- Presentation Attacks / Face Replay* (Uso de foto estática ou vídeo para burlar a câmera).
- Cross-IdP Impersonation* (Sequestro de CPF usando logins sociais de terceiros).
- Registro remoto indevido ou automação via scripts.
- Compartilhamento abusivo de dispositivos.

Este RFC define os mecanismos de mitigação em camadas para o MVP, alinhados estritamente à LGPD e à arquitetura de Identidade Canônica Centralizada.

2. Princípios Antifraude

- Nunca confiar exclusivamente no client:** O frontend (React) atua apenas como coletor; o veredito ocorre na máquina servidora.
- Descarte da Mídia Fonte (Diretriz LGPD):** Mídias brutas (fotos da face) não são armazenadas em disco para validação de presença; persistem apenas os hashes biométricos.
- Comando Único e Imutabilidade:** O registro bruto gerado pelo cliente é imutável, o risco é

classificado posteriormente.

4. **Validação Estendida na Fronteira (2x2):** O vínculo inicial da biometria exige orquestração governamental (fé pública). O registro diário usa o motor interno.
 5. **Defesa em Profundidade (*Defense-in-Depth*):** Múltiplas barreiras de análise (Localização, Biometria, Dispositivo e Comportamento).
-

3. Modelo de Ameaças

3.1 GPS Spoofing

Usuário simula localização usando aplicativos falsos (Mock Locations) ou manipulando a API de geolocalização do navegador.

3.2 Face Replay e Liveness Spoofing

O fraudador apresenta fotografias de alta resolução, telas de celular ou máscaras para a webcam tentando simular a presença física do titular.

3.3 Sequestro de Identidade (Cross-IdP Impersonation)

Um invasor realiza login com um e-mail falso via Keycloak e, na tela de cadastro, tenta informar o CPF de um terceiro para assumir sua folha de ponto.

3.4 Manipulação de Timestamp

Tentativa de alterar o horário local do dispositivo para fraudar o horário de entrada ou saída.

3.5 Device Sharing Abusivo

Um único usuário utiliza seu dispositivo pessoal para registrar o ponto de diversos outros colaboradores de forma orquestrada (Modo Kiosk não autorizado).

4. Arquitetura Antifraude — As 5 Camadas

- **Camada 1 — Validação Geográfica** (PostGIS)
 - **Camada 2 — Validação Biométrica em 2 Níveis** (Governamental + Motor Interno)
 - **Camada 3 — Integridade e Confiança de Dispositivo** (Device Fingerprinting)
 - **Camada 4 — Análise Comportamental e Risco Composto** (Heurísticas)
 - **Camada 5 — Auditoria Centralizada e Imutável** (Logs e Envers)
-

5. Geolocalização (Camada 1)

5.1 Decisão Arquitetural

- O polígono oficial da unidade (`poligono_geo`) reside na tabela

`folha_ponto.unidade`.

- O cálculo geodésico (verificação se o ponto está contido na área) é executado unicamente pelo PostGIS no PostgreSQL.
- O evento bruto na tabela `folha_ponto.registro_presenca` salva as coordenadas soltas (latitude, longitude) e a `precisao_gps`.

5.2 Validações no MVP

1. **Precisão:** Se a `precisao_gps` informada pelo hardware for superior ao raio limite configurado, o evento sofre acréscimo de pontuação de risco.
2. **Distância e Continência:** O backend processa se a coordenada intercepta a geometria da unidade daquele evento.
3. **Saltos Impossíveis:** Comparação da distância percorrida desde o último ponto em relação ao tempo decorrido.

5.3 Mitigação de Mock Location

- O frontend deve consultar APIs nativas para detectar uso de localizações simuladas (quando aplicável) e injetar essa suspeita no payload.
 - O backend registrará no campo `indicadores_risco` (JSONB) da tabela `registro_presenca` a flag indicando o uso de ferramentas de *spoofing*.
-

6. Biometria Facial e Liveness (Camada 2)

A biometria obedece a dois momentos de validação distintos, resolvendo a vulnerabilidade do Sequestro de Identidade.

6.1 Onboarding e Criação do Vínculo (Validação 2x2)

- Para impedir o *Cross-IdP Impersonation*, a primeira amarração do CPF exige uma **Validação 2x2 (Biográfica + Biométrica)**.
- O Spring Boot enviará a face capturada e o CPF informados no React para um provedor de fé pública (ex: Serpro / Datavalid).
- Apenas após aprovação governamental, o Hash Biométrico é extraído, e a tabela `folha_ponto.perfil_biometrico` é populada.

6.2 Registro Diário (Motor Python Interno)

- Para o controle de presença rotineiro, o backend envia a requisição para o motor Python local via API interna.
- O motor interno:
 1. Executa avaliação de Prova de Vida (*Liveness Detection*) avaliando profundidade 3D na matriz do frame para bloquear *Face Replay*.
 2. Gera o template biométrico.
 3. Compara o vetor extraído com o vetor armazenado na tabela `perfil_biometrico`.
- O resultado é guardado no banco através dos campos `pontuacao_biometrica` (o *score*

numérico) e `biometria_valida` (booleano baseado em *threshold*).

6.3 O Princípio do Descarte da Mídia Fonte (Compliance LGPD)

Em conformidade com a LGPD, o arquivamento das fotografias rasterizadas (os arquivos `.jpg/.png` brutos da "selfie") é estritamente proibido. A imagem capturada permanece restrita à memória RAM efêmera do servidor Java/Python durante os milissegundos necessários para a extração do vetor matemático (Hash Biométrico) e é imediatamente varrida pelo *Garbage Collector*. No banco de dados, armazena-se unicamente o vetor resultante na coluna `modelo_biometrico` da tabela `perfil_biometrico`.

7. Integridade de Dispositivo (Camada 3)

7.1 Identificação (Device Fingerprinting)

O sistema exige a gestão de *hardware* confiável através da tabela `folha_ponto.dispositivo`.

- O frontend gera e envia um UUID de rastreo (`identificador_dispositivo`).
- O backend consulta/grava o dispositivo no banco e verifica o seu histórico de uso.
- A tabela `registro_presenca` atrela obrigatoriamente o evento à chave estrangeira `dispositivo_id`.

7.2 Nível de Confiança

A tabela de dispositivo comporta o campo `nivel_confianca` (LOW, MEDIUM, HIGH). Esse nível é degradado automaticamente pelo backend se o mesmo aparelho físico tentar registrar a presença para dezenas de `usuario_id` diferentes fora do modo Kiosk oficial.

7.3 Modo de Registro

Para preparar a base de dados para Kiosks ou tablets de parede, a tabela de eventos suporta a coluna `modo_registro`.

- Valores previstos: `SELF` (Aparelho próprio) ou `SHARED_DEVICE` (Aparelho compartilhado homologado).
-

8. Timestamp Seguro (Camada 4)

O sistema opera com dupla checagem temporal para evitar fraudes de manipulação de relógio do celular. A tabela `registro_presenca` exige obrigatoriamente duas colunas:

1. `capturado_em`: O horário alegado pelo dispositivo cliente.
2. `recebido_no_servidor_em`: O horário real e blindado da máquina servidora (em UTC).

Se a divergência entre os dois tempos ultrapassar o limite aceitável de rede, o evento é marcado

com alto `pontuacao_risco`. A regra de negócio para cálculos de jornada utilizará primordialmente a hora do servidor.

9. Score de Risco Composto

Cada requisição de presença passará por um algoritmo de *Risk Scoring* que compilará os fatores das camadas anteriores, gerando uma consolidação armazenada na coluna `pontuacao_risco` (float) e detalhada no campo `indicadores_risco` (JSONB).

Fórmula referencial: `pontuacao_risco = f(geofence_distance, gps_accuracy, biometric_score, liveness_flag, device_trust_level, temporal_divergence)`

No MVP, o sistema adotará a política de Comando Único, focando no registro bruto imutável. Registros com alta `pontuacao_risco` recebem um `status_administrativo` de PENDENTE e exigirão tratamento gerencial (exceções).

10. Auditoria Imutável (Camada 5)

Como última linha de defesa, a trilha de segurança é garantida pela arquitetura de banco de dados (RFC-0005):

1. **A Regra de Ouro:** O evento bruto de `registro_presenca` nunca é deletado. Qualquer reclassificação do evento (ex: "registro redundante") apenas altera o seu `status_administrativo` ou `status_tecnico`, preservando a evidência da fraude.
 2. **Log Centralizado:** A tabela `folha_ponto.log_auditoria` será utilizada para registrar formalmente as tentativas de acesso negadas pela Validação 2x2 do Governo (Serpro), violações de regras de negócio graves do motor de políticas e banimentos de dispositivos.
-

11. Arquitetura Técnica Recomendada

O processamento das camadas 1 a 4 deve ser implementado no Spring Boot através de um módulo ou *pipeline* de interceptadores apartado da regra de negócio de ponto em si (ex: `FraudDetectionEngine`). O motor de políticas avaliará os resultados gerados por este módulo e decidirá se emite a exceção ou apenas *tagueia* o evento com risco alto.

12. Riscos Aceitáveis no MVP vs. Inaceitáveis

- **Risco Aceitável:** *Spoofing* sofisticado baseado em interceptação a nível de *kernel* (root/jailbreak) disfarçado perfeitamente que entregue coordenadas e imagens injetadas.
- **Risco Inaceitável:** Validação de *geofence* confiada apenas ao JavaScript do React.

- **Risco Inaceitável:** Armazenamento da foto facial do servidor em disco para fins de biometria.
-

13. Decisão Estratégica

A arquitetura estabelece que as validações antifraude mais caras e complexas (Prova de Vida governamental via Validação 2x2) ocorram na "fronteira" (Onboarding), ancorando firmemente o CPF de forma irrefutável. Para as milhares de transações diárias de ponto, o sistema confia em heurísticas de retaguarda (Geofence, Motor Python Local e Device ID) para equilibrar extrema segurança com baixa latência e economia de custos de API.